

Índice

-  [Módulo 1 — Introducción a DevOps](#)
-  [Módulo 2 — Herramientas \(Git, GitHub, Docker\)](#)
-  [Módulo 3 — CI/CD](#)
-  [Checklist rápido para el parcial](#)

Módulo 1 — Introducción a DevOps

1.1 ¿Qué es DevOps?

Definición (del PDF + clase):

DevOps es la **práctica en la que los ingenieros de operaciones y desarrollo participan juntos en todo el ciclo de vida del servicio**, desde el diseño, pasando por todo el proceso de desarrollo, hasta la implementación y el soporte de producción.

- **"Dev"** → todas las personas involucradas en el desarrollo del producto (front, back, Producto, QA).
- **"Ops"** → ingenieros de sistemas, SysAdmins, operaciones, release engineers, DBAs, redes, seguridad.

Lucas en Clase 2

"DevOps **no es un puesto de trabajo**, es una **cultura**. Confunden DevOps con SRE. Busquen en LinkedIn y van a ver 'DevOps' como título, pero como tal **no debería existir** como rol aislado."

Origen → DevOps surgió de la colisión de dos tendencias:

1. **Infraestructura Ágil u Operaciones Ágiles** (aplicar Agile y Lean a operaciones).
2. **Colaboración ampliada Dev ↔ Ops** en todo el ciclo de vida.

1.2 Los 4 Pilares: CAMS

Culture · Automation · Measurement · Sharing

Pilar	Descripción
Culture (Cultura)	Romper silos entre Dev y Ops. Colaboración, comprensión mutua, trabajo en equipo. Lo fundamental : va más allá de las comodidades físicas.
Automation (Automatización)	Eliminar trabajo manual. Debe equilibrarse con procesos y personas . Solo se automatiza cuando el proceso ya está bien definido.
Measurement (Medición)	Métricas en toda la organización: tiempo de recuperación, ciclos, satisfacción de empleados, revenue, costo.
Sharing (Compartir)	Transparencia, apertura, intercambio de conocimientos. Si un equipo resuelve algo, lo comparte con el resto.

Lucas en clase

Los términos fueron acuñados por **John Willis** y **Damon Edwards**. A veces se ve como **CALMS** (agregando Lean), pero lo original es CAMS.

"Una de las cosas es que **Kaizen enfatiza la importancia de ir al lugar real** donde se crea valor, no solo confiar en métricas."

1.3 The Three Ways (Las Tres Vías)

1. **System Thinking (Pensamiento Sistémico)** → Comprender el sistema en su totalidad, desde el concepto hasta la entrega, en lugar de optimizar partes individuales.
2. **Feedback Loops (Bucles de Retroalimentación)** → Amplificar los ciclos de feedback.
3. **Continuous Experimentation and Learning** → Cultura de experimentación y aprendizaje continuo.

¿Cuándo aplicar The Three Ways?

- Para **crear procesos** en el equipo
- Para **crear estándares** en el equipo
- Como parte de un **estilo de management**

Concepto asociado: From Concept to Cash

Flujo desde el concepto/idea hasta el producto final monetizado. Si la organización no puede entregar software de manera efectiva al cliente → el esfuerzo se pierde.

1.4 Lean (7 Principios)

Del método Lean del Sistema de Producción de **Toyota** (1986):

1. **Eliminación de desperdicio**
2. **Amplificación del aprendizaje**
3. **Toma de decisiones lo más tarde posible**
4. **Entrega rápida**
5. **Empoderamiento del equipo**
6. **Integridad incorporada**
7. **Ver el todo** (visión end-to-end del proceso)

Kaizen (改善) — "Cambio para mejor"

- **Mejora continua** con pequeños cambios que generan grandes resultados.
- Ciclo: **planificación** → **ejecución** → **análisis** → **acción** (similar al método científico).
- Herramienta clave: **los 5 porqués** para llegar a la raíz de un problema.
- Enfocado en **causas subyacentes**, evita culpar al error humano.
- Promueve **pensamiento crítico** en las personas.
- Principio: **ir al lugar real** (Gemba) donde se crea valor, no solo mirar métricas.

Build, Measure, Learn

Metodología de **Eric Ries** en *Lean Startup*:

Construir → **Medir** → **Aprender** (MVP → retroalimentación → iteración).

1.5 Agile / Manifiesto Ágil (2001)

Valores clave:

- **Individuos e interacciones** sobre procesos y herramientas
- **Software funcionando** sobre documentación extensiva
- **Colaboración con el cliente** sobre negociación contractual
- **Respuesta ante el cambio** sobre seguir un plan

 **Lucas en clase**

"Antes del 2000, sin Internet, la documentación era todo un manual. A partir del 2001 con Internet, estos cambios se pudieron dar mejor."

1.6 🏆 10 Buenas Prácticas para el Éxito (DevOps)

#	Práctica	Descripción
1	Chaos Monkey	Herramienta que provoca fallos aleatorios en la infraestructura para probar la resiliencia del sistema.
2	Blue-Green Deployment	Dos entornos idénticos: uno activo (azul) y otro inactivo (verde). Se despliega al inactivo y se redirige tráfico.
3	Dependency Injection	Patrón para dependencias débilmente acopladas . La lógica de negocio no debe conocer infraestructura ni BD. Base de las arquitecturas limpias.
4	Andon Cords	Mecanismos visuales/automáticos que alertan sobre problemas. Viene de Toyota : cualquier operario podía tirar de la cuerda y parar toda la producción para que todos analizaran el problema.
5	Feature Flags	(Derivado — ver CI/CD) Permiten activar/desactivar funcionalidades sin nuevos deploys.
6	(Continúa en clase)	
7	Blameless Postmortems	Reuniones post-incidente para analizar causas sin culpar a individuos o equipos. Publicados por empresas como Amazon.
8	Status Pages	Interfaces públicas que muestran estado y disponibilidad en tiempo real (ej. status de Amazon, Slack, Azure).
9	Developers on Call	Desarrolladores asignados para responder a incidentes fuera del horario laboral (rotativo).
10	Incident Command System (ICS)	Sistema estandarizado para coordinar respuesta a incidentes, desde pequeños hasta grandes catástrofes. Primera línea → Segunda línea → Tercera línea (postmortem).

Otras prácticas mencionadas

- **Embedded Team (Equipos multidisciplinarios)** → Dev, Ops, QA y Seguridad **juntos** en el mismo equipo. Evita que seguridad aparezca al final y bloquee el deploy.
- **Cloud** → Escalabilidad, flexibilidad, agilidad para desplegar.

1.7 DevOps Especializado (evoluciones)

DevOps se extendió a dominios específicos manteniendo los pilares CAMS:

- **GitOps** → Infra como Código + Git como fuente de verdad (Alexis Richardson, 2017).
Herramientas: ArgoCD, Flux.
 - **MLOps** → Automatización del ciclo de ML, versionado de modelos, detección de drift.
 - **DevSecOps** → Seguridad integrada. **Shift Left**: detectar problemas **antes** de producción.
SAST, Dependency Scanning, Container Scanning, IaC Security, Secret Management.
 - **DataOps, FinOps, PlatformOps** → Otras especializaciones.
-



Módulo 2 — Herramientas (Git, GitHub, Docker)

2.1 Tools Overview

Las tres herramientas que vemos:

- **Git** → Sistema de **control de versiones**.
 - **GitHub** → Plataforma de **hosting y colaboración**.
 - **Docker** → Plataforma de **virtualización de contenedores**.
-

2.2 Git — Version Control System (VCS)

¿Qué es?

Un **VCS** gestiona la evolución de archivos en el tiempo y conserva historial completo de cambios.

Objetivos principales:

- **Trazabilidad** de quién cambió qué y cuándo
- **Colaboración segura** entre múltiples personas
- **Recuperación** de versiones estables ante errores
- Soporte a **branching, merging y release management**

Beneficios:

- Menor riesgo de pérdida de trabajo

- Auditoría técnica del proyecto
- Más velocidad para experimentar y corregir

Tipos de VCS

Tipo	Características
Local	Solo en tu máquina
Centralizado	Servidor central único (ej. SVN). Dependés del servidor.
Distribuido (Git)	Cada desarrollador tiene copia completa del repositorio. Permite trabajar offline.

Main Features de Git

- **Sistema distribuido** — cada dev tiene el repo completo
- **Rendimiento** — rápido con grandes proyectos
- **Funcionalidades** — branches, tags, merges
- **Flexibilidad** — se adapta a diferentes flujos

Git Workflow (flujo típico)

```

Working Directory → Staging Area → Local Repository → Remote
Repository
    (editar)           (git add)       (git commit)       (git push)
  
```

Comandos Git esenciales

```

git init           # Inicializa un repo local
git clone <url>    # Clona un repo remoto
git add . / git add <file> # Prepara cambios para commit
git commit -m "mensaje" # Registra cambios en repo local
git status         # Estado actual del working tree
git log --oneline  # Resumen del historial
git pull           # Trae y fusiona cambios del remoto
git push           # Publica commits locales al remoto
  
```

Conventional Commits

Formato:

```
<tipo>(scope opcional): descripción
```

Tipos comunes:

- `feat` : nueva funcionalidad
- `fix` : corrección de bug
- `docs` : cambios de documentación
- `chore` : tareas de mantenimiento
- `refactor` : mejora interna sin cambiar comportamiento

Ejemplos:

```
git commit -m "feat(auth): agregar login con OAuth"
git commit -m "fix(api): corregir timeout en endpoint de usuarios"
git commit -m "docs(readme): actualizar guía de instalación"
git commit -m "chore(deps): actualizar dependencias de seguridad"
```

Beneficios: Historial más legible · Changelogs automáticos · Mejor trazabilidad para CI/CD.

Commit Message Best Practices

- Usar **verbo en imperativo**: add, fix, remove, update
- Una **idea por commit** (cambios pequeños y atómicos)
- Título breve y claro (≤ 72 caracteres)
- Explicar **qué** y cuando haga falta **por qué**
- Evitar mensajes genéricos como "changes", "fix", "update stuff"

Git Tags y Releases

Un **tag** es una referencia fija a un commit específico.

Usos:

- Marcar versiones de release (`v1.0.0` , `v1.1.0`)
- Facilitar **rollback** a versiones conocidas
- Asociar artefactos y notas de lanzamiento

```
git tag v1.0.0
git push origin v1.0.0
git checkout v1.0.0
```

"Junto con los conventional commits, los tags nos permiten **automatizar el resumen de cambios por versión** (changelog automático). Además, asociamos artefactos como la app o librería a ese punto del historial."

Branches (Ramas)

Las ramas permiten:

- Trabajar en **versiones aisladas** sin afectar el código productivo
- Agregar funcionalidades o correcciones en paralelo
- Después hacer **merge** hacia la rama principal

2.3 Branching Strategies

Trunk-Based Development

- **Cambios pequeños y frecuentes** sobre `main`
- Ramas de **vida muy corta** o integración directa con **feature flags**
- **Cuándo usar:** equipos con **fuerte automatización de CI/CD**

GitHub Flow

1. Todo lo que está en `main` está **listo para producción**
2. Para algo nuevo → crear rama desde `main` con nombre descriptivo
3. Integrar trabajo en esa rama local y regularmente en servidor
4. Cuando se necesite ayuda / rama lista → abrir **Pull Request**
5. Alguien revisa y visa cambios → merge a `main`
6. Los cambios integrados se ponen en producción

Cuándo usar: equipos pequeños/medianos con releases frecuentes.

GitFlow

- Estructura **rígida** para releases planificados
- Ramas dedicadas: `feature`, `release`, `hotfix`, `develop`, `main`
- Flujo: `feature` → `develop` → `release` → `main` (+ `hotfix`)
- **Cuándo usar:** releases periódicos (ej. cada 6 meses, como el kernel de Linux)

"Head Flow [GitHub Flow] es más recomendado para **equipos muy pequeños o proyectos chicos**. GitFlow funciona bien con ciclos de releases cada cierto tiempo."

2.4 GitHub

Key Features

- **Repositorios** — alojar proyectos con control de versiones Git
- **Colaboración** — trabajo en equipo y code reviews
- **Issues** — seguimiento de problemas, tareas, mejoras
- **Pull Requests** — revisión y aprobación de cambios
- **Integraciones** — amplia gama de herramientas/servicios
- **Wikis y documentación**

First Steps on GitHub

1. Crear repositorio en GitHub
2. Clonar localmente con `git clone`
3. Hacer cambios y crear commits en una rama
4. Publicar rama con `git push`
5. Abrir una **Pull Request** para revisión e integración

Pull Request Example

```
git checkout -b feature/monitoring-basics
git add .
git commit -m "docs: add monitoring basics slide"
git push -u origin feature/monitoring-basics
```

Luego en GitHub:

1. Crear Pull Request hacia `main`
2. Solicitar review
3. Hacer merge al aprobarse

Conventional Comments (para code reviews)

Prefijos sugeridos:

- **nitpick**: sugerencia menor
- **suggestion**: propuesta de mejora
- **question**: consulta o duda
- **praise**: reconocimiento positivo
- **issue**: problema a corregir

Ejemplo: `suggestion`: Considera extraer esta lógica a una función aparte para mayor claridad.

2.5 Docker

¿Qué es?

Plataforma de **virtualización por contenedores** lanzada en **2013**. Revolucionó el trabajo en la nube y con aplicaciones modernas.

Docker vs Virtual Machines

Aspecto	Contenedores Docker	Virtual Machines
Qué es	Emulación de espacio de usuario	Emulación de máquina física completa
Virtualización	Abstrae detalles del SO	Abstrae detalles del hardware
Administrado por	Docker Engine	Hipervisor
Arquitectura	Comparte kernel del host	Kernel privado por VM
Recursos	Bajo demanda	Cantidad fija
Kernel	Compartido (namespaces + cgroups)	Propio

🔗 Lucas en Clase 3

"Las máquinas virtuales tienen **mayor consumo** porque emulan toda una máquina con el mismo hardware. Los contenedores son **virtualización a nivel del SO**: comparten el kernel del host. Son **más ligeros y eficientes**."

Ventajas sobre VMs

- **Rapidez:** arranque en milisegundos (no segundos)
- **Overhead bajo:** megabytes, no gigabytes de RAM
- **Densidad:** cientos/miles por máquina física
- **Portabilidad:** misma imagen = mismo comportamiento en todos lados
- **CI/CD ágil:** más fácil de construir, testear y distribuir
- **Escalabilidad horizontal:** spin up/down en segundos

Desventaja: todos comparten el kernel Linux → no tan aislados como VMs si hay vulnerabilidad en el kernel.

Conceptos básicos

- **Imagen** → plantilla **inmutable versionada**. Contiene app + dependencias.
- **Contenedor** → **instancia en ejecución** de una imagen. Puede haber varios del mismo tipo.
- **Dockerfile** → archivo con instrucciones para **construir** la imagen.
- **Registry** → almacén central de imágenes (Docker Hub, registros privados).

Docker Architecture

Componentes:

- **Docker Client** — interfaz CLI (`docker` commands)
- **Docker Daemon** — servicio de fondo que gestiona contenedores e imágenes
- **Docker Images** — plantillas versionadas e inmutables
- **Docker Containers** — instancias aisladas en ejecución
- **Docker Registry** — almacén central

Flujo típico:

1. Cliente envía comando al Daemon
2. Daemon verifica/descarga imagen del Registry
3. Daemon crea y ejecuta contenedor

Docker Image Layers

Cada layer es una capa de cambios sobre la anterior:

```
FROM nginx:alpine          # capa base
RUN apt-get install curl    # nueva capa
```

```
COPY . /app                # nueva capa
ENTRYPOINT ["nginx"]       # configuración
```

Ventajas:

- **Reutilización** de capas (reduce almacenamiento)
- **Caché de construcción** (builds más rápidos)
- **Reproducibilidad** garantizada

Docker Example

```
FROM nginx:alpine
COPY . /usr/share/nginx/html
EXPOSE 80
```

```
docker build -t demo-web:1.0 .
docker run --rm -p 8080:80 demo-web:1.0
```

Docker Data Volumes

Permiten **persistencia** o **compartir datos** entre host y contenedor.

Tipos:

- **Bind mount** — mapea directorio host al contenedor (`-v /host:/container`)
- **Named volume** — volumen gestionado por Docker
- **Anonymous volume** — temporal, se borra con el contenedor

Cuándo usar:

- Persistir base de datos (nunca en `/tmp` del contenedor)
- Compartir código host ↔ contenedor en desarrollo
- Separar datos (stateful) de la aplicación (stateless)

Use Cases de Docker

- **Despliegue de aplicaciones:** encapsular app + dependencias, portabilidad.
 - **Entornos de desarrollo:** consistentes y reproducibles, elimina "en mi máquina funciona".
 - **Pruebas y QA:** entornos aislados y controlados.
 - **Microservicios:** cada módulo en su propio contenedor, escalado independiente.
-

2.6 DevOps Workflow (End-to-End)

Cómo se conectan Git, GitHub y Docker en un flujo real:

1. **Desarrollar** cambios en local
2. **Versionar con Git** (`add` , `commit`)
3. **Publicar rama y abrir PR** en GitHub
4. **Construir imagen Docker** de la aplicación
5. **Desplegar la misma imagen** en diferentes entornos

Diagrama:

Code Change → Git Commit → Push Branch → Pull Request → Merge to main → Docker Build → Deploy



Módulo 3 — CI/CD

3.1 ¿Qué es CI/CD?

CI/CD = Continuous Integration + Continuous Delivery/Deployment

Es el **core técnico** del DevOps moderno. Se representa con el símbolo del infinito (∞) que une Dev (Code, Build, Test) con Ops (Release, Deploy, Operate, Monitor).

3.2 Continuous Integration (CI)

Definición y Objetivos

Definición: Práctica de **automatizar la integración** de los cambios de código de varios contribuyentes en un único proyecto de software de forma **frecuente** (mínimo una vez al día).

Objetivos principales:

1. **Detectar errores lo antes posible** (Fail Fast)
2. **Reducir conflictos de integración** ("Integration Hell")
3. **Mantener una base de código siempre estable**

🔗 Lucas en Clase 4

"La idea es simple: trabajar en ramas aisladas por días o semanas, al final todos integran en la principal. Si alguien introduce un error, CI nos avisa. Se evitan errores en el

momento del merge, que **con 10, 30 o 50 personas después de una semana es muchísimo laburo.**"

CAMS en CI/CD

Pilar	Aplicación en CI/CD
Culture	Colaboración y transparencia en el proceso de entrega
Automation	El core del pipeline
Measurement	Pruebas automáticas y monitoreo de performance
Sharing	Repositorios compartidos y visibilidad total de despliegues

Principio clave

"Si duele, hazlo más seguido" — Si integrar código es difícil, **hazlo cada hora, no cada mes**. La frecuencia reduce la complejidad.

3.3 Etapas del CI

1 CODE

Todo comienza con el **control de versiones** (SCM):

- **Version Control:** Git es el estándar
- **Branching Strategies:** Trunk-based (ramas cortas, integración diaria) vs GitFlow (releases planificados)
- **Code Review:** Pull Requests para asegurar calidad con ojos humanos
- **Herramientas:** GitHub, GitLab, Bitbucket

2 BUILD

El sistema **compila y prepara** los binarios/artefactos.

Herramientas CI: GitHub Actions, CircleCI, Travis CI, Jenkins.

3 TEST — "Nuestra red de seguridad"

Tipo de test	Qué prueba
Unit Tests	Lógica interna de funciones
Integration Tests	Cómo se comunican mis módulos entre sí
Static Analysis (Linting)	Calidad de código y estilo
Security (SAST)	Análisis estático de vulnerabilidades

Herramientas: Jest (JS), JUnit (Java), PyTest (Python), SonarQube, Snyk, Testify (Go).

Code Coverage

Algoritmo que mide qué líneas ejecutó el test. Sirve como herramienta, pero no es 100% preciso (un test puede pasar por el código sin probarlo realmente).

3.4 Ejemplo: GitHub Actions Workflow

```
name: Node.js CI
on: [push, pull_request]

jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Setup Node
        uses: actions/setup-node@v3
        with: { node-version: '18' }
      - run: npm install
      - run: npm test
      - run: npm run build
```

Estructura:

- `name` → nombre del workflow
- `on` → eventos que lo disparan (push, pull_request)
- `jobs` → trabajos a ejecutar
- `runs-on` → imagen del runner (ubuntu-latest, corre sobre contenedor Docker de Ubuntu)
- `steps` → pasos secuenciales

Lucas en clase

"El `runs-on: ubuntu-latest` es **como un contenedor de Docker** corriendo sobre Ubuntu. Hacemos checkout del código dentro de ese contenedor, después `npm install`, luego los tests (unit, integration) y el linter."

3.5 Continuous Delivery vs Continuous Deployment

La diferencia clave

- **Continuous Delivery (Entrega Continua):** el código está **siempre listo** para ser desplegado, pero el paso final a producción se decide **manualmente** (botón "Aprobar").
- **Continuous Deployment (Despliegue Continuo):** **todo es automático**. Si el código pasa las pruebas, llega a producción automáticamente.

¿Todas las empresas tienen CD automatizado?

NO. Razones mencionadas en clase:

1. **Cultura** — resistencia organizacional al cambio.
2. **Entornos regulados** — negocio prefiere decidir cuándo lanzar (marketing, timing comercial).
3. **Madurez técnica** — CD requiere confianza total en tests y herramientas.
4. **Costo/complejidad** — automatizar completo puede ser costoso.

Lucas en Clase 4

"Muchos equipos **tratan de llegar al despliegue continuo**. Siempre se comienza primero por la **entrega continua**. El despliegue continuo requiere **madurez y confianza total** en el suite de pruebas."

3.6 Etapas de CD: Release & Deployment

Release

- **Versionado:** SemVer (Semantic Versioning: MAJOR.MINOR.PATCH)
- **Release Notes:** notas de lanzamiento con todos los cambios
- **Artifact Registry:** almacenamiento de los paquetes

Note

Un release **no es solo código**: incluye paquete, versionado y documentación (release notes).

Deployment

El acto de **poner el código en el servidor/cloud**.

3.7 Deployment Strategies

Canary Deployment

Origen del nombre: En las minas, los mineros llevaban un **canario en una jaula**. Mientras el canario estuviera vivo, el aire era seguro. Si el canario moría → salir inmediatamente.

Cómo funciona:

1. Desplegar nueva versión solo al **5% de usuarios**
2. Monitorear errores
3. Si no hay errores → progresar (5% → 10% → 20% → 50% → 100%)
4. Generalmente **3-4 etapas** como máximo
5. Dejar tiempo entre etapas para uso real (no 10 min)

Blue-Green Deployment

Dos entornos **idénticos**:

- **Azul** (activo) — versión vieja
- **Verde** (inactivo) — nueva versión

Se van migrando usuarios del azul al verde gradualmente. **Cero tiempo de inactividad, fácil rollback.**

Note

En realidad pueden ser **clusters completos** (ej. en Kubernetes), no una sola instancia.

Rolling Deployment

Actualiza **parte de los servidores** o instancias a la vez. Mantiene la app disponible durante el proceso.

Despliegue Progresivo

Cambios graduales con **mayor control** sobre infraestructura y usuarios (filtros por usuarios, configuraciones).

Rollback Automático

No es exactamente una **deployment strategy**, pero es esencial:

- Si se detectan errores durante el deploy → **revertir automáticamente** a versión anterior
- Muchos procesos de deployment ya vienen con esto integrado

3.8 Operate & Monitoring

El ciclo **no termina** cuando el código "está vivo":

- **Operate**: Infraestructura como Código (IaC), escalado automático, gestión de secretos
- **Monitoring**: Observabilidad → **Logs, Métricas, Traces**
- **Feedback Loop**: Datos de producción informan planificación futura

Herramientas: Kubernetes, Prometheus, Grafana, ELK, Terraform.

3.9 CI/CD Pipeline Seguro (DevSecOps)

Escaneos en cada stage:

```
Code Commit → SAST → Secrets Scan → Dependency Scan (CVEs) → Container Scan  
→ Deploy
```

↓

⚠ Vulnerability Found

✓ Checklist rápido para el parcial

Módulo Introducción

- ☐ Definición de DevOps (cultura, no rol)
- ☐ Origen: Infra Ágil + colaboración Dev-Ops
- ☐ **CAMS**: Culture, Automation, Measurement, Sharing
- ☐ **Three Ways**: System Thinking, Feedback Loops, Continuous Experimentation
- ☐ **Lean**: 7 principios + Kaizen + 5 porqués + Build-Measure-Learn
- ☐ **Agile**: 4 valores del Manifiesto
- ☐ **10 Prácticas**: Chaos Monkey, Blue-Green, Dependency Injection, Andon Cords, Feature Flags, Blameless Postmortems, Status Pages, Developers on Call, ICS
- ☐ Embedded Teams + Cloud

Módulo Herramientas

- ☐ **VCS**: trazabilidad, colaboración, recovery, branching
- ☐ **Git**: distribuido vs centralizado, comandos clave
- ☐ **Git Workflow**: Working Dir → Staging → Local → Remote
- ☐ **Conventional Commits**: <tipo>(scope): desc (feat/fix/docs/chore/refactor)
- ☐ **Tags & Releases** (SemVer)
- ☐ **Branching**: Trunk-based vs GitHub Flow vs GitFlow
- ☐ **GitHub**: repos, PRs, issues, flow completo
- ☐ **Docker vs VMs** (kernel compartido vs hypervisor)
- ☐ **Docker**: imagen, contenedor, Dockerfile, registry
- ☐ **Docker Architecture**: Client, Daemon, Images, Containers, Registry
- ☐ **Image Layers** y cache
- ☐ **Volumes**: bind mount, named, anonymous
- ☐ **Use Cases** de Docker

Módulo CI/CD

- ☐ Definición CI y sus 3 objetivos
- ☐ "Si duele, hazlo más seguido"
- ☐ Etapas CI: **Code** → **Build** → **Test**

- ☐ Tipos de tests: Unit, Integration, Static Analysis, SAST
 - ☐ GitHub Actions (estructura YAML)
 - ☐ **Continuous Delivery vs Deployment** (diferencia clave)
 - ☐ Por qué no todas las empresas tienen CD automatizado
 - ☐ **Release**: SemVer + Release Notes + Artifact Registry
 - ☐ Estrategias de despliegue: **Canary** (origen del nombre), **Blue-Green**, **Rolling**, Progresivo
 - ☐ **Rollback** automático
 - ☐ Operate & Monitoring: IaC, observabilidad, feedback loop
-

Referencias

- **PDFs del módulo:** `intro.pdf`, `DevOps_Introduction.pdf`, `DevOps_Tools.pdf`, `CI_CD.pdf`, `04deployment.pdf`
 - **Transcripciones:** Clase 1, Clase 2, Clase 3, Clase 4
 - **Lecturas recomendadas (Lucas):**
 - Martin Fowler — *Continuous Integration*
 - Gene Kim & Jez Humble — *The DevOps Handbook*
 - John Willis — *DevOps Culture, The Andon Cord*
 - John Allspaw — *Blameless Postmortems and a Just Culture*
 - Alexis Richardson — *GitOps - Operations by Pull Request*
 - Google — *SRE Book*
-

✓ Tip de estudio

Lucas suele preguntar por **definiciones** y **relaciones entre conceptos** (ej: cómo Kaizen se refleja en CAMS, por qué Canary se llama así, diferencia Delivery vs Deployment). No te limites a memorizar: **entendé los "por qué"**.